TEAM 16

Intent-Based SDN for 5G Service-Level Agreement Enforcement

Team 16 - Development Progress Report

REPORT DATE

2026-08-30

DELIVERY STATUS

5 of 14 subtasks complete

EVIDENCE BASIS

Live git, pytest, Ruff, demo, repository inventory, and line-count commands

Prepared as an evidence-backed engineering status document. No live-network result is claimed.

1. Executive summary

This system lets an operator write a machine-readable promise about a network connection, including bandwidth, latency, and loss. The software validates that intent, computes a path that satisfies its constraints, and translates the result into OpenFlow rules for enforcement. The intent pipeline works end to end in software: validation, persistence, path search, constraint pruning, deterministic selection, and enforcement rule generation are covered by the current test suite. Live-network execution begins at ST-5, but it has not yet been verified on hardware, Mininet, or a live Open vSwitch datapath.

Current verified position: 172 tests pass and Ruff reports no findings across src, tests, topology, and experiments. The git history contains completed milestone commits from ST-1 through ST-5.

2. What was built

Line counts below are physical line counts collected from the current working tree. They are shown to make scope visible, not as a productivity metric.

Subtask	Delivered capability	Current files and physical line counts
ST-1	Container and grammar foundation. Python 3.9 and Ryu 4.34 are explicitly pinned. The Dockerfile installs Mininet and Open vSwitch from Ubuntu 22.04 apt without exact package pins, so the requested Mininet 2.3.1b4 and OVS 2.17 claim is not verified by the current repository. The frozen JSON Schema contains 5 constraint types, 3 violation actions, and priority bounds 1 through 5. The inventory contains 8 scenario intents, 6 invalid examples, continuous integration, and 3 ADRs.	Dockerfile (40 lines) docker-compose.yml (21 lines) requirements.txt (20 lines) .github/workflows/ci.yml (41 lines) src/intent_manager/schema.json (118 lines) src/intent_manager/validator.py (224 lines) docs/decisions.md (90 lines)
ST-2	Topology specification with 7 switches, 8 hosts, and 3 link-disjoint paths from s1 to s7; Mininet topology construction; B0 baseline harness; flow-table pilot; and architecture document.	topology/topology_spec.py (162 lines) topology/team16_topo.py (136 lines) experiments/scenarios/b0_baseline.sh (34 lines) experiments/scenarios/flow_table_pilot.sh (16 lines) docs/architecture.md (110 lines) tests/test_topology.py (181 lines) tests/test_topo_script.py (117 lines)
ST-3	Frozen shared interfaces: intent and path models, typed event bus, SQLite persistence schema, and an audit log that preserves the causal chain from measurement to action.	src/common/models.py (369 lines) src/common/events.py (206 lines) src/common/db.py (399 lines) src/common/audit.py (146 lines) tests/test_common.py (385 lines)
ST-4	M1 REST API and M3 path computation using Yen k-shortest paths, constraint pruning, readable rejection reasons, and deterministic best-path selection.	src/intent_manager/api.py (189 lines) src/pathing/yen_ksp.py (105 lines) src/pathing/prune.py (134 lines) src/pathing/compute.py (92 lines) tests/test_pathing_and_api.py (334 lines) demo_st4.py (132 lines)
ST-5	M4 enforcement: OpenFlow 1.3 rule generation, deterministic cookies, OVS HTB queue command construction, a Ryu controller, and the Scenario S1 shell workflow. This code is unit-tested without Ryu, Mininet, root, or real ovs-vsctl execution.	src/enforcement/flowmod.py (269 lines) src/enforcement/queues.py (107 lines) src/enforcement/controller.py (219 lines) experiments/scenarios/s1_single_intent.sh (110 lines) tests/test_enforcement.py (435 lines)

3. Architecture

The design separates intent interpretation, feasibility, routing, enforcement, observation, and explanation. Each module is accountable for one operational question.

Module	Responsibility	Question answered
M1	Intent management	What does the user want?
M2	Admission control	Can we provide it?
M3	Path computation	How can we provide it?
M4	Network enforcement	Can we make the network do it?
M5	Telemetry and detection	Is it actually working?
M6	Dashboard and audit	What is happening and why?

Topology alternatives

Route	Bottleneck	One-way delay
s1 -> s2 -> s3 -> s7	100 Mbps	6.0 ms
s1 -> s4 -> s5 -> s7	50 Mbps	15.0 ms
s1 -> s6 -> s7	40 Mbps	24.0 ms

Three link-disjoint paths are mandatory because a closed-loop controller needs a materially different route when the active path violates an SLA or loses a link. Without alternatives there is nothing to re-plan to, so rerouting behavior and recovery metrics cannot be tested.

4. Evidence of working software

The following blocks are captured directly by the report builder. They are proof of software behavior, not live-network evidence.

python demo_st4.py - captured output

```
================================================================================
1. THE NETWORK
================================================================================
7 switches, 8 links

   s1-s2       100 Mbps     2.0 ms
   s2-s3       100 Mbps     2.0 ms
   s3-s7       100 Mbps     2.0 ms
   s1-s4        50 Mbps     5.0 ms
   s4-s5        50 Mbps     5.0 ms
   s5-s7        50 Mbps     5.0 ms
   s1-s6        40 Mbps    12.0 ms
   s6-s7        40 Mbps    12.0 ms

   Routes from s1 to s7 (where h1 and h2 live):
   s1 -> s2 -> s3 -> s7      100 Mbps     6.0 ms
   s1 -> s4 -> s5 -> s7       50 Mbps    15.0 ms
   s1 -> s6 -> s7             40 Mbps    24.0 ms

================================================================================
2. A VALID INTENT IS ACCEPTED AND COMPILED TO A PATH
================================================================================
intent:
  id: INT-001
  tenant: telemedicine-video
  priority: 1
  match: {src: 10.0.0.1/32, dst: 10.0.0.2/32, proto: udp, dport: 5001}
  guarantee: {min_bandwidth_mbps: 20, max_latency_ms: 20, max_loss_pct: 0.5}

   validation : accepted

   promise    : 20 Mbps, 20 ms, 0.5% loss
   chosen path: s1 -> s2 -> s3 -> s7
   delivers   : 100.0 Mbps, 6.0 ms
   links used : s1-s2, s2-s3, s3-s7
   computed in: 0.8 ms  (budget is 200 ms)

================================================================================
3. AN IMPOSSIBLE INTENT IS REFUSED, WITH A REASON
================================================================================
intent:
  id: INT-002
  tenant: bulk-backup
  priority: 4
  match: {src: 10.0.0.7/32, dst: 10.0.0.8/32, proto: tcp, dport: 873}
  guarantee: {min_bandwidth_mbps: 500, max_latency_ms: 5}

   path found : None
   why not    :
     s3-s7                 insufficient bandwidth: 100 < 500
     s3-s2-s1-s4-s5-s7     exceeds max_latency_ms: 19 > 5
     s3-s2-s1-s6-s7        exceeds max_latency_ms: 28 > 5

   No silent failure. Every refusal explains itself.

================================================================================
4. THE INTENT GRAMMAR IS FROZEN
================================================================================
intent:
  id: INT-003
  tenant: rogue

  priority: 9
  match: {src: 10.0.0.1/32, dst: 10.0.0.2/32}
  guarantee: {max_jitter_ms: 5}

   validation : rejected
     intent.guarantee
       Additional properties are not allowed ('max_jitter_ms' was unexpected)
     intent.priority
       9 is greater than the maximum of 5

   priority 9 is outside the 1-5 scale, and max_jitter_ms is not one
   of the five frozen constraint types. Both are caught before the
   intent reaches the network.

================================================================================
5. STATE IS PERSISTED
```

```
================================================================================
   schema version : 1
   intents stored : 0
   current path   : s1 -> s2 -> s3 -> s7
   reroute count  : 1  (metric E6)

================================================================================
STATUS
================================================================================
   Built   : ST-1 stack freeze, ST-2 topology, ST-3 interfaces,
             ST-4 intent API and path computation
   Next    : ST-5 install these paths as real OpenFlow rules
   Not yet : nothing touches a live network. That starts at ST-5.
```

pytest tests -q - captured summary

```
.................................................................... [ 41%]
.................................................................... [ 83%]
............................                                         [100%]
172 passed, 2 warnings in 4.75s
```

5. Verification

Test inventory by file

Test file	Collected cases
tests/test_common.py	32
tests/test_enforcement.py	39
tests/test_intent_validation.py	50
tests/test_pathing_and_api.py	32
tests/test_topo_script.py	7
tests/test_topology.py	12
Total	172

Lint status

No findings; command exited successfully.

Git commit evidence

Commit	Date	Subject
b9553d0	2026-08-30	Add evidence-backed PDF progress report builder
14eb622	2026-08-30	ST-5: M4 enforcement - OpenFlow rules, HTB queues, Ryu controller, S1 scenario
2803d6b	2026-08-30	Add demo_st4.py: one-command walkthrough of everything built so far
0baebbb	2026-08-30	ST-4: M1 REST API and M3 path computation
7259c48	2026-08-30	ST-3: drop unused import flagged by ruff
2ab45eb	2026-08-30	ST-3: freeze the shared interfaces
953b0f3	2026-08-30	ST-2: topology script, B0 baseline, flow-table pilot, architecture doc
e1e1a31	2026-08-30	ST-2 review: add acceptance tests for the topology script itself
20eeae9	2026-08-30	ST-2 (spec side): topology spec, acceptance tests, implementation brief
ca2f078	2026-08-21	ST-1: pin stack, freeze intent grammar, scaffold repo

6. Honest limitations

1. Nothing has been executed on Mininet or Open vSwitch yet; docker compose build has not been run.

2. ST-5 enforcement code is written but unverified against a live datapath.

3. No novelty is claimed; this is a reference implementation and evaluation.

4. Metric E5 may return a flat result because software switches have no TCAM scarcity; a pilot is scheduled to confirm.

5. The current Dockerfile does not pin exact Mininet or Open vSwitch package versions; it installs the Ubuntu 22.04 apt packages.

7. Remaining work

Subtask	Outcome
ST-6	Telemetry collection and probes
ST-7	Close the feedback loop
ST-8	Admission control
ST-9	Preemption and hysteresis
ST-10	Two-phase versioned updates
ST-11	Dashboard and operator explanation
ST-12	Evaluation phase one
ST-13	Evaluation phase two
ST-14	Final report

ST-12 and ST-13 are protected evaluation time. If implementation slips, cut features, never measurement. The value of the project depends on defensible evidence rather than an expanded but unevaluated feature set.

Evidence note: this report was generated from live repository commands on 2026-08-30. The working suite returned 172 passing tests; the earlier 133-test expectation is superseded by the added ST-5 enforcement coverage.